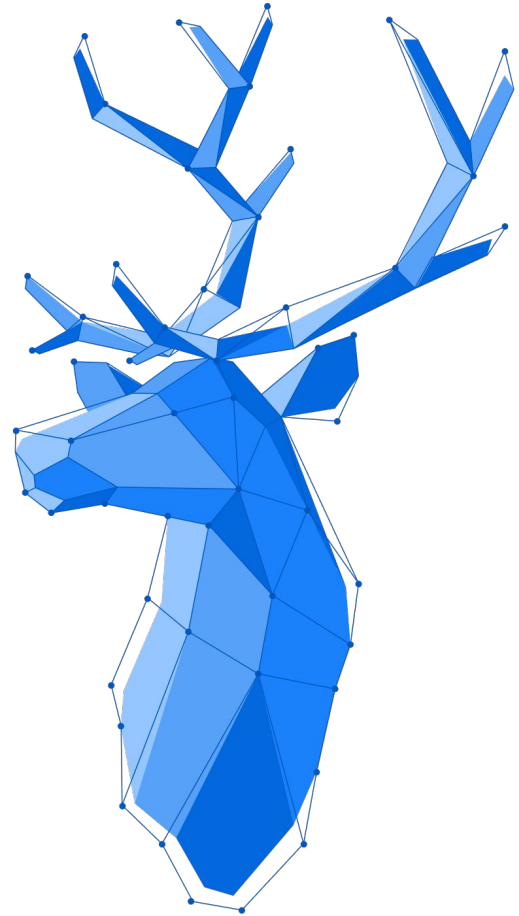




Binder

Von Gerald Koch



Einleitung

- Möglichkeit Formulare ohne viele Listener an Beans zu binden
- Kann Werte validieren
- Kann werte konvertieren
- Kann direkt und lazy verwendet werden
- Kann indirekt geschachtelt werden

Unterschied Direktes VS Lazy Binding

Direkt

- Jede Änderung eines Feldes direkt in die assoziierte Bean geschrieben wird.
- Verwendet `void setBean(T)` zum Setzen der Bean
- Verwendet `T getBean()` zum holen der Bean

Lazy (Indirekt)

- Indirektes Binding bedeutet, dass jede Änderung eines Feldes innerhalb des Binders gespeichert wird und die Bean während des Bearbeitungsvorganges nicht angetastet wird.
- Verwendet `void readBean(T)` zum Setzen der Bean
- Verwendet `writeBean(T)` zum holen der Bean
- Änderungen können durch erneutes Lesen der Bean rückgängig gemacht werden

Verschiedene Möglichkeiten

Der Vaadin Binder kann auf verschiedene Arten verwendet werden:

- Automatisches Property Binding
 - Keine Konverter
 - Kann beschränkt Validatoren auf Bean-Ebene mittels Annotationen haben
 - `new Binder<Bean>(Bean.class)`
- Manuelles Property Binding
 - Voller Funktionsumfang
 - `Binder binder = new Binder<Bean>();`
`binder.forField(x).bind("property");`
- Getter/Setter Binding (empfohlen)
 - Voller Funktionsumfang
 - `Binder binder = new Binder<Bean>();`
`binder.forField(x).bind(Bean::getX, Bean::setX);`

Binder

Konverter

Konverter werden verwendet um Feld-Datentypen in Bean-Datentypen umzuwandeln. Zum Beispiel können Telefonnummern geparkt und in ihre bestandteile Landesvorwahl, Ortsvorwahl und Durchwahl aufzusplitten oder um eine List zu einem Set umzuwandeln.

Beispiel List ↔ Set

```
static class Bean {
    private List<Task> tasks;
    public List<Task> getTasks()
    {return tasks;}
    public void setTasks(List<Task> tasks)
    {this.tasks = tasks;}
}

Binder<Bean> binder = new Binder<>();
// Grid multi select works with Set
binder.forField(taskGrid.asMultiSelect())
    .withConverter(
        c -> (List<Task>) new ArrayList<>(c), // to Bean
        LinkedHashSet::new) // to Field (Grid)
    .bind(Bean::getTasks, Bean::setTasks);
```

Validatoren

Validatoren werden verwendet, um Eingabewerte zu prüfen. Es gibt verschiedene von Vaadin bereitgestellte Validatoren:

- StringLengthValidator
- RegexpValidator
 - EmailValidator
- RangeValidator
 - LongRangeValidator
 - DoubleRangeValidator
 - DateRangeValidator
 - DateTimeRangeValidator
 - u.v.m.

Validatoren

- Validatoren können zur Wiederverwendung implementiert werden
 - Interface `Validator` implementieren
 - Klasse `AbstractValidator` überschreiben
- Validatoren können auch als kurzes Lambda oder innere Klasse an die Methode `withValidator` übergeben werden
- Vaadin Komponenten bringen ihren eigenen Fehlertext-Platzhalter
- `Binder.setStatusLabel(HasText)` legt eine Komponente zum Anzeigen der Binder-Fehler fest
- `Binder.setValidationStatusHandler(...)` kann dafür genommen werden, um Feldvalidierungen umzuleiten